

Yazılım Test Türleri

Kapsamlı Referans Kılavuzu

Ömer Faruk Yavuz

December 2025

Temel Fonksiyonel Testler

Unit Test

Tek bir fonksiyon veya sınıfın, tüm dış bağımlılıklar (DB, HTTP, dosya sistemi) **mock'lanarak** izole test edilmesi. Local geliştirme sırasında sürekli (`npm test` gibi) ve her PR açıldığında CI'da koşar.

Örnek: `calculateDiscount(price, isPremium)` fonksiyonu için:

`calculateDiscount(100, false) → 100` (indirim yok) `calculateDiscount(100, true) → 80` (%20 indirim)

Bu testlerde veritabanı çağrısı ve HTTP isteği yoktur; yalnızca girdi/çıktı doğrulanır.

Checklist: Önemli if/else dalları kapsanıyor · tüm dış bağımlılıklar mock'lanmış · test isimleri açıklayıcı (`should_apply_discount_for_premium_users`) · deterministik (rastgele başarısız olmuyor)

Mülakat: “Unit test, dış bağımlılıkları mock'layarak tek bir fonksiyon veya sınıfın davranışını izole test eder; her PR'da hızlıca koşup iş mantığını korumaya yardımcı olur.”

Integration Test

Birden fazla bileşenin (Controller + Service + Repository + DB, Service + Redis vb.) **birlikte** doğru çalışıp çalışmadığını test eder. Büyük refactor veya kritik değişikliklerde geliştirici tarafından; CI'da unit testlerden sonra koşar.

Örnek: `POST /users` endpoint testi: Docker'da Postgres ayağa kaldırılır; istek atılır; beklenenler: HTTP status 201, response body'de `id` ve `email`, DB'de gerçekten kullanıcı kaydı oluşmuş.

Checklist: Testler gerçek (test) DB ile koşuyor · her test kendi verisini kurup temizliyor · hem happy path hem en az bir hata senaryosu var · CI'da deterministik

Mülakat: “Integration test, birden fazla bileşenin gerçeğe yakın ortamda birlikte doğru çalışıp çalışmadığını doğrular; kritik endpoint'ler için mutlaka kullanırım.”

API Test

Backend endpoint'lerinin **HTTP seviyesinde** doğru davranıp davranmadığını test eder: status kodları, response body, hata formatları, auth davranışı. CI'da PR sonrası ve staging'de deploy sonrası koşar.

Örnek: `POST /auth/login`: geçerli credential → 200 + accessToken; yanlış şifre → 401 + anlamlı hata mesajı. `GET /users/:id`: auth varken → 200 + kullanıcı bilgileri; auth yoksa → 401; kayıt yoksa → 404.

Checklist: Ana endpoint'lerde happy path var · auth gereken yerlerde unauthorized (401) test edildi · validation hataları (400) için test case var · response şemaları doğrulandı

Mülakat: “API testleri, backend endpoint'lerinin giriş/çıkış davranışını HTTP seviyesinde doğrular; özellikle auth ve kritik domain endpoint'leri için bu testlere çok önem veririm.”

Contract Test

Mikroservis mimarisinde servisler arası **API sözleşmesinin bozulmamasını** sağlar. Consumer servis beklediği şemayı tanımlar; provider bu şemayı karşılıyor mu diye CI pipeline'ında her iki tarafta ayrı ayrı test edilir.

Örnek: Consumer: `GET /payments/:id` için beklenen JSON şeması tanımlanır. Provider: payment

servisi bu şemayı gerçekten döndürüp döndürmediği açısından test edilir. Şema kırılırsa merge engellenir.

Checklist: Kritik endpoint'ler için contract tanımı mevcut · CI'da hem provider hem consumer tarafında koşuyor · contract kırılırsa merge engelleniyor veya güçlü uyarı üretiliyor

Mülakat: "Contract test, servisler arası API sözleşmesinin bozulmamasını sağlar; bir servisin yaptığı değişikliklerin diğerlerini kırmaması için kullanılır."

End-to-End (E2E) Test

Gerçek kullanıcı gibi UI → Backend → Veritabanı → (varsa) 3. taraf servisler zincirini uçtan uca test eder. Genellikle staging ortamında; bazı kritik senaryolar CI'da headless olarak da koşabilir.

Örnek: Randevu sistemi için tipik bir E2E akışı:

1. Kullanıcı kayıt olur.
2. E-posta doğrulama adımı tamamlanır (mock veya test inbox).
3. Kullanıcı login olur.
4. Yeni randevu oluşturur.
5. Randevu liste ekranında yeni randevuyu görür.

Checklist: Yalnızca 3–5 kritik kullanıcı akışı E2E ile test ediliyor · flaky değil; CI pipeline'ını gereksiz yere kırmıyor · test hesapları kullanılıyor · toplam süre makul (10–15 dk)

Mülakat: "E2E testler, gerçek kullanıcı senaryolarını UI'dan başlayarak uçtan uca doğrular; sayısı az ama iş açısından en kritik akışlar için kullanırım."

Doğrulama Testleri

UAT — User Acceptance Test

Geliştirilen özelliğin iş gereksinimlerini karşılayıp karşılamadığını product owner, iş birimi veya müşteri temsilcisiyle staging'de doğrular. Her user story için **acceptance criteria** yazılır; story ancak bu kriterler sağlandığında "done" sayılır.

Örnek: "Kullanıcı geçmiş 30 güne ait randevuları tarih sırasına göre görebilmeli" kriteri, UAT sırasında ürün yöneticisi tarafından staging'de adım adım kontrol edilir.

Checklist: Her story için net acceptance kriterleri yazılmış · UAT sırasında her kriter gözden geçirilmiş · küçük UX/UI düzeltmeleri not alınmış · sonuçlar Jira/Confluence'da kayıt altında

Mülakat: "Acceptance/UAT testleri, geliştirilen özelliğin iş gereksinimlerini gerçekten karşıladığını doğrulamak için product veya müşteriyle birlikte yapılan testlerdir."

Smoke Test

Her deploy sonrası sistemin **en temel fonksiyonlarının** çalıştığını çok hızlı kontrol eder. "Makine yanıyor mu, yanmıyor mu?" sorusunun cevabını arar. Staging'e her deploy sonrası otomatik; büyük release'lerde prod'a çıkmadan önce de koşulur.

Otomatik smoke test için tipik endpoint'ler:

- GET /health → status 200, body {"status":"ok"}
- GET /version → status 200, body {"version":"...", "commit":"..."}
- POST /auth/login (test kullanıcısı) → status 200 + accessToken
- Ana domain endpoint'lerinden 1–2 tane (ör. GET /appointments)

Manuel smoke checklist:

Checklist: Uygulama açılıyor, ana sayfa hata vermeden yükleniyor · test kullanıcısıyla login olunabiliyor · ana iş akışı baştan sona çalışıyor · ağır hata mesajları veya crash yok

Mülakat: "Smoke test, release sonrası sistemin temel fonksiyonlarının ayakta olduğunu hızlıca doğrulayan basit test setidir; health check, login ve ana domain endpoint'leri genellikle bu sette yer alır."

Sanity Test

Küçük veya orta büyüklükte bir değişiklikten sonra, tam regresyon yapmadan ilgili fonksiyonun **mantıklı çalışıp çalışmadığını** hızlıca kontrol eder. Geliştirici veya QA tarafından staging'de

yapılır.

Örnek: Görev listesinde sıralama mantığı değişmiştir. Sanity test: görevler beklenen sırada mı? Filtreler hâlâ çalışıyor mu? İlgili ekranların temel fonksiyonları bozulmamış mı?

Checklist: Değişiklik yapılan ekranın ana senaryoları denendi · bağlı 1–2 ekranın temel fonksiyonları bozulmadı · kritik hata veya crash gözlenmedi

Mülakat: “Sanity test, belirli bir değişiklikten sonra tam regresyon yapmadan ilgili fonksiyonların mantıklı çalıştığını hızlıca doğrular.”

Regression Test

Yeni özellik veya bug fix sonrası **önceden çalışan fonksiyonların bozulup bozulmadığını** kontrol eder. CI’da unit + integration + seçili E2E; büyük release’ler öncesinde daha geniş manuel + otomatik set koşulur.

Örnek: Kritik happy path’ler (sign up, login, ödeme, ana iş akışı) yeniden test edilir; geçmişte bulunan önemli bug’lar için yazılmış testler çalıştırılarak tekrar çıkmaları engellenir.

Checklist: Kritik ana akışlar regresyon seti içinde yer alıyor · önemli geçmiş bug’lar için regression test yazılmış · CI pipeline’ında düzenli koşuyor

Mülakat: “Regression test, yeni değişikliklerin mevcut çalışan fonksiyonellikleri bozmadığını doğrulamak için daha geniş kapsamlı bir test setini tekrar çalıştırmaktır.”

Performans Testleri

Load Test

Sistemin **beklenen normal yük altında** nasıl davrandığını ölçer. Örneğin aynı anda 500 kullanıcı varken ortalama gecikme süresi ve hata oranı ölçülür. Araçlar: k6, JMeter, Locust, Gatling.

Checklist: Response time hedeflerin altında · hata oranı eşik değerin altında · sistem beklenen yük altında stabil · kaynak kullanımı (CPU, RAM) normal

Mülakat: “Load test, sistemin beklenen normal yük altında tepki süresi ve hata oranını ölçer.”

Stress Test

Sistemin kapasitesinin **üstüne çıkarak** nerede ve nasıl bozulduğunu gösterir. Yük kademeli artırılır: 500 → 1.000 → 2.000 eş zamanlı istek. Kırılma noktası, ilk ciddi hata oranı ve toparlanma süresi kayıt altına alınır.

Checklist: Kırılma noktası (breaking point) tespit edildi · sistem nasıl ve nerede bozuluyor belli · toparlanma davranışı izlendi · ölçeklenme limitleri belgelendi

Mülakat: “Stress test, sistemi kapasitesinin üstüne yükleyerek nerede ve nasıl bozulduğunu görmek için yapılır.”

Soak / Endurance Test

Sistemi uzun süre (örneğin 6–24 saat) boyunca sabit bir yük altında tutarak **memory leak, connection leak, kaynak tükenmesi** gibi uzun vadeli problemleri ortaya çıkarır.

Checklist: Uzun süreli testte memory kullanımı artmıyor · connection pool tükenmesi yok · hata oranı zaman içinde artmıyor · CPU kullanımı stabil

Mülakat: “Soak test, sistemi uzun süre boyunca sürekli yük altında tutarak hafıza sızıntısı ve kaynak tükenmesi gibi uzun vadeli problemleri ortaya çıkarmayı hedefler.”

Spike Test

Trafik veya istek sayısı **çok kısa sürede aniden arttığında** (ör. kampanya başlatma, viral içerik) sistemin buna verdiği tepkiyi test eder. Yük bir anda 10x–50x artırılır, sistem çökmeden ayakta kalabiliyor mu gözlemlenir.

Checklist: Spike sırasında sistem çökmüyor · otomatik ölçeklendirme devreye giriyor · spike sonrası sistem normale dönüyor · kullanıcı deneyimi kabul edilebilir düzeyde

Mülakat: “Spike test, sistemin ani trafik artışlarına verdiği tepkiyi ölçer.”

Scalability Test

Sistem yük arttıkça (100 → 1.000 → 10.000 kullanıcı) performansın nasıl değiştiğini ve yatay/dikey ölçeklemenin ne kadar etkili olduğunu ölçer. CPU, memory, response time ve error rate izlenir.

Checklist: Response time yük arttıkça kabul edilebilir seviyede · servisler otomatik ölçeklenebiliyor · kaynak kullanımı optimize

Mülakat: “Scalability test, sistem yük arttığında performansın nasıl değiştiğini ve ölçeklemenin ne kadar verimli olduğunu ölçer.”

Volume Test

Sistemin **çok büyük veri kümeleri** (milyonlarca kayıt) altında nasıl çalıştığını test eder. DB’ye büyük miktarda test verisi yüklenir; API performansı ve query hızları ölçülür.

Checklist: Büyük veri altında query süreleri kabul edilebilir · API time-out vermiyor · DB kaynak tüketimi izleniyor

Mülakat: “Volume test, sistemin çok büyük veri kümeleriyle nasıl performans gösterdiğini ölçer.”

Concurrency Test

Birden fazla kullanıcı **aynı kaynağa aynı anda eriştiğinde** race condition veya veri tutarsızlığı olup olmadığını test eder. Transaction, optimistic/pessimistic locking davranışları incelenir.

Örnek: Aynı ürünü aynı anda düzenlemeye çalışan 10 paralel istek; stok miktarının yanlış güncellenmesi veya kayıt çakışması izlenir.

Checklist: Race condition gözlenmiyor · lock mekanizmaları doğru çalışıyor · veri kaybı yaşanmıyor

Mülakat: “Concurrency test, ortak kaynağa aynı anda erişimde sistemin race condition’a karşı dayanıklı olup olmadığını doğrular.”

Güvenlik ve Dayanıklılık Testleri

Security / Penetration Test

Güvenlik açıklarını (yetkisiz erişim, veri sızıntısı, SQL injection, XSS, CSRF vb.) tespit eder.

Araçlar:

- Otomatik: dependency vulnerability taramaları (Snyk, npm audit), OWASP ZAP benzeri scanner’lar.
- Manuel: güvenlik uzmanları tarafından hedefli saldırı senaryoları (OWASP Top 10 esas alınır).

Mülakat: “Security/penetration testleri, sistemde yetkisiz erişim veya veri sızıntısına yol açabilecek güvenlik açıklarını bulmak için yapılan testlerdir; genelde OWASP risklerine odaklanılır.”

Reliability Test

Sistemin uzun süre boyunca hatasız çalışabilme oranını ölçer. Error log, crash raporu, uptime ve error rate metrikleri izlenir.

Checklist: Uzun süreli çalışmada hata oranı stabil · memory artışı yok/minimum · servis uptime değeri kabul edilebilir

Mülakat: “Reliability test, sistemin uzun süre boyunca hatasız çalışabilme yeteneğini ölçer; uptime gerektiren sistemlerde kritiktir.”

Recovery / Failover Test

Çökme, servis kesintisi veya donma yaşandığında sistemin kendini ne kadar hızlı ve doğru toparladığını, veri kaybı veya bozulması yaşanıp yaşanmadığını test eder. Sunucu kasıten durdurulur, network kesilip geri verilir, DB restart senaryoları uygulanır.

Checklist: Servis kesinti sonrası otomatik toparlanıyor · veri kaybı minimum · retry mekanizmaları doğru çalışıyor

Mülakat: “Recovery test, çökme veya servis kesintisi sonrası sistemin kendini toparlama yeteneğini doğrular.”

Chaos Test (Chaos Engineering)

Üretim benzeri ortamda bilinçli arızalar çıkararak sistemin dayanıklılığını ölçer: rastgele sunucu kapatma, network latency ekleme, CPU/RAM baskısı oluşturma. Araçlar: Chaos Monkey, Gremlin.

Checklist: Servis kapanınca diğer servisler fallback'e geçiyor · trafik doğru yönlendiriliyor · sistem tamamen çökmüyor

Mülakat: "Chaos test, üretim benzeri ortamda bilinçli arızalar yaratarak sistemin dayanıklılığını ölçer ve SRE kültürünün temel parçalarındandır."

Network Partition Test

Dağıtık sistemlerde (Kafka, Redis Cluster, Postgres Patroni) network bölünmesi (split-brain) yaşandığında servislerin davranışını test eder. Leader election, eventual consistency, retry ve timeout mekanizmaları doğrulanır.

Checklist: Partition sırasında servisler çalışmaya devam ediyor · partition bitince sistem toparlıyor (reconciliation) · veri kaybı veya duplicate event oluşmuyor · retry/timeout davranışları doğru

Mülakat: "Network partition test, dağıtık sistemlerde bağlantı bölünmesi yaşandığında sistemin tutarlılığını ve dayanıklılığını doğrular."

Kullanıcı Deneyimi ve Uyumluluk Testleri

Usability Test

5–10 gerçek kullanıcıya görevler verilerek uygulamanın ne kadar rahat ve sezgisel kullanıldığı ölçülür. UX problemleri (kafa karışıklığı, fazla adım, kötü yerleşim) bulunur. Yeni ekran tasarımından sonra ve beta release öncesi yapılır.

Örnek: Kullanıcılara "Randevu oluştur", "Filtreyi bul", "Favoriye ekle" gibi görevler verilir; nerede takıldıkları gözlemlenir.

Not ("Taze Kan" Etkisi): Ürüne alışmış ekip zamanla kurumsal körlük yaşar. Yeni kullanıcılar bu kör noktaları doğal etkileşimleriyle çok hızlı ortaya çıkarır.

Checklist: Kullanıcı görevleri 30–60 saniyede tamamlayabiliyor · navigation gereksiz karmaşık değil · UI elementleri anlaşılır · kullanıcı geri bildirimleri toplandı

Mülakat: "Usability test, gerçek kullanıcıların uygulamayı ne kadar kolay ve sezgisel kullandığını ölçer; UX problemlerini erken yakalar."

Heuristic Evaluation

1–3 UX uzmanı, ekranları Nielsen'in 10 heuristiği (tutarlılık, görünürlük, kullanıcı kontrolü, hata engelleme, minimalist tasarım vb.) gibi standart UX prensiplerine göre inceler ve sorunları listeler.

Checklist: Ekran yapısı tutarlı · hatalar kullanıcıyı engellemez · navigasyon kullanıcı zihniyle uyumlu · UI elementleri sezgisel ve yeterince görünür

Mülakat: "Heuristic evaluation, kullanıcı arayüzünü bilinen UX ilkelerine göre uzman gözle değerlendirerek kullanılabilirlik sorunlarını erken yakalar."

Accessibility Test

Engelli kullanıcılar için WCAG standartlarına uyum doğrulanır. Screen reader (TalkBack/VoiceOver) testleri, renk körlüğü simülatörleri ve kontrast oranı ölçümleri kullanılır.

Checklist: Tüm butonların erişilebilirlik label'ları var · kontrast oranı 4.5:1 altında değil · odak sırası mantıklı · TalkBack ile ekran tamamen okunabiliyor

Mülakat: "Accessibility test, engelli kullanıcıların uygulamayı sorunsuz kullanabilmesini sağlamak için yapılan erişilebilirlik doğrulamasıdır."

Compatibility Test

Uygulamanın farklı cihazlarda, tarayıcılarda (Chrome, Safari, Firefox, Edge) ve işletim sistemlerinde doğru çalıştığını doğrular. Araçlar: BrowserStack, LambdaTest.

Checklist: En az 3 farklı ekran boyutunda test edildi · son 2 büyük OS sürümü destekleniyor · ana

fonksiyonlar tüm tarayıcılarda çalışıyor

Mülakat: “Compatibility test, uygulamanın farklı cihaz ve tarayıcılarda aynı kaliteyle çalıştığını doğrular.”

A/B Test

İki farklı tasarım, akış veya özellik arasında kullanıcıların bir kısmı A, diğerleri B versiyonunu görür; click rate, conversion ve drop-off oranları karşılaştırılarak hangi versiyonun daha iyi performans verdiği ölçülür.

Checklist: Deneme yeterli kullanıcı sayısına ulaştı · istatistiksel anlamlılık (p-value) değerlendirildi

Mülakat: “A/B test, iki farklı tasarım veya akışın performansını karşılaştırarak en iyi seçeneği belirler.”

Localization / Internationalization Test

Uygulamanın farklı dil (TR/EN/DE/AR), tarih formatı, para birimi, RTL dilleri ve bölgesel ayarlarla doğru çalıştığını doğrular.

Checklist: Metin taşması yok · RTL modunda ekran tasarımı bozulmuyor · tarih/para formatları doğru

Mülakat: “Localization test, uygulamanın çok dilli ortamlarda ve farklı kültürel ayarlarda doğru çalıştığını doğrular.”

Mobil Özel Testler

Mobile-Specific: Network, Background, Permissions

- **Network:** 3G → 4G → WiFi geçişlerinde session kopuyor mu? Offline mod davranışı doğru mu?
- **Background:** Uygulama arka plana alınca state korunuyor mu? Timer, konum, audio recorder gibi süreçler devam ediyor mu?
- **Permissions:** Kamera/konum/mikrofon izni reddedilince doğru fallback ekranı geliyor mu?

Checklist: Offline mod test edildi · background/foreground geçişlerinde state kaybı yok · izin reddedilince kullanıcı dostu hata gösteriliyor

Mülakat: “Mobil özel testler; ağ değişimleri, background davranışı ve izin yönetimi gibi mobil cihazlara özgü riskleri doğrular.”

Monkey Test

Uygulamayı rastgele tıklamalar, kaydırmalar ve kontrolsüz girişlerle kırmaya çalışılır. Gerçek hayattaki hatalı kullanıcı davranışlarını simüle eder. Mobil uygulamalarda (özellikle Android) yaygındır; release öncesi hızlı stabilite kontrolü.

Örnek: Android: `adb shell monkey -p com.example.app 5000` (uygulamaya 5000 rastgele dokunuş, swipe, rotate, back action)

Web: rastgele tıklamalar üreten custom script'ler.

Checklist: 5k–20k event ile çalıştırıldı · crash report veya ANR çıktı mı? · navigation loop veya freeze yaşandı mı? · log'lar incelendi ve hata stack'leri kaydedildi

Mülakat: “Monkey test, rastgele kullanıcı davranışlarıyla uygulamayı zorlayarak crash ve stabilite sorunlarını ortaya çıkarır.”

Gorilla Test

Tek bir fonksiyonu aşırı yükleyerek veya tekrar tekrar çalıştırarak kararlılığı test eder. Bir butona 1000 kez tıklama simülasyonu, aynı formu 100 defa ardışık gönderme, upload bileşenine art arda yük bindirme.

Checklist: Seçilen modülde 500–2000 tekrar yapıldı · memory artışı/leak gözlemlendi mi? · UI freeze oluştu mu? · concurrency davranışı incelendi

Mülakat: “Gorilla test, belirli bir fonksiyonu yüzlerce kez tekrar ederek stabilite sorunlarını tespit etmeye odaklanır.”

Battery Usage Test

Mobil uygulamanın gereksiz pil tüketip tüketmediğini ölçer. Konum, kamera, mikrofon veya sürekli senkronizasyon kullanan uygulamalarda kritiktir.

- **Android:** Battery Historian ile CPU, wakelock, network kullanım raporları.
- **iOS:** Xcode Instruments “Energy Log” ile enerji tüketim profili.
- Uygulama 10–20 dakika foreground ve background modlarında test edilir.

Checklist: CPU kullanımı anormal değil · wakelock sayısı düşük ve kontrollü · gereksiz sensör (GPS, accelerometer) kullanımı yok · background modunda pil tüketimi minimum

Mülakat: “Battery test, mobil uygulamanın pil tüketimini analiz eder; konum, kamera ve background görevleri olan uygulamalarda kritik bir kalite göstergesidir.”

Cold Start Test

Uygulamanın RAM’de yokken (soğuk başlatma) ilk açılış hızını ölçer: Splash ekranından ilk interaktif ekrana kadar geçen süre optimize edilmiş mi?

- **Android:** `adb shell am force-stop com.app` ile uygulama kapatılır; açılış süresi ölçülür.
- **iOS:** App switcher’den tamamen kapatılır; Xcode Instruments Time Profiler ile profillenir.

Ölçülen metrikler: startup time · first paint · first interactive · JS engine (Hermes/JSCore) load süresi · module initialization.

Checklist: Splash → ilk ekran geçişi 1–2 saniye içinde · gereksiz modül açılışta yüklenmiyor · bundle boyutu optimize · başlangıçta ağır network isteği yok

Mülakat: “Cold start test, uygulamanın tamamen kapalı durumdan ilk açılış hızını ölçer; mağaza kalite metrikleri için kritik bir performans göstergesidir.”

Keşif ve Fuzzing Testleri

Exploratory Test

Testçi belirli bir modül veya akış üzerinde serbestçe dolaşarak öğrendikçe yeni test fikirleri üretir ve uygular. Ad-hoc’tan farklı olarak yönlendirilmiş ve deneyim odaklıdır; öğrenme, tasarlama ve uygulama aşamaları iç içe yürür.

Checklist: Temel akışlar keşfedildi · beklenmeyen davranışlar not alındı · her adım için gözlem ve sonuç kaydedildi

Mülakat: “Exploratory test, testçinin deneyimiyle yön verdiği, öğrenme ve keşif aşamalarının iç içe yürüdüğü sistematik bir keşif testidir.”

Ad-hoc Test

Plansız, spontane şekilde hata bulmak; dokümantasyon olmadan keşif yapmak. Özellikle ilk prototip aşamalarında kullanılır.

Örnek: Edge-case denemeleri: çok uzun input, boş input, hızlı tıklama, offline/online geçişi, aynı anda birden fazla sekme açmak.

Checklist: Kritik ekranlar rastgele keşfedildi · anormal davranışlar not alındı · en az 1 tane UI bug’ı yakalanmaya çalışıldı

Mülakat: “Ad-hoc test, plansız şekilde uygulamayı keşfederek beklenmeyen davranışları ortaya çıkarma testidir.”

Fuzz Test

Sisteme rastgele, bozuk, ekstrem veya beklenmeyen inputlar göndererek parser, validator ve hata yönetimi hatalarını ve güvenlik açıklarını bulur. API endpoint’lerine yüzlerce/binlerce hatalı istek gönderilir; JSON, XML, form alanları bozuk formatlarla manipüle edilir.

Checklist: Beklenmeyen inputlar sistem crash’ine yol açmıyor · API 400/422 gibi anlamlı hata döndürüyor · validation tüm alanlar için devrede

Mülakat: “Fuzz test, API veya parser gibi bileşenlere rastgele ve bozuk input göndererek beklenmeyen

hata ve güvenlik açıklarını bulmaya yarar.”

Yaşam Döngüsü ve Yapılandırma Testleri

Alpha Test

Ürün dışı açılmadan önce şirket içinde (QA ekibi ve geliştiriciler) büyük hataların temizlenmesi amacıyla yapılan ilk geniş kapsamlı test.

Checklist: Kritik bug'lar temizlendi · UI/UX akışları kararlı

Mülakat: “Alpha test, ürün dışı kullanıcıya açılmadan önce şirket içinde yapılan ilk geniş kapsamlı testtir.”

Beta Test

Seçili gerçek kullanıcılara beta sürümü dağıtılır; gerçek ortam geri bildirimleri toplanır ve ürün iyileştirmelerine aktarılır.

Checklist: Beta kullanıcılarından geri bildirim toplandı · kritik hatalar düzeltildi

Mülakat: “Beta test, gerçek kullanıcıların ürünü deneyerek geri bildirim sağladığı ve yayından önce hataların yakalandığı ikinci aşama testidir.”

Installation / Deployment Test

Uygulamanın doğru kurulum başlatılabildiğini doğrular. Mobil: temiz cihazda ilk açılış testi; Back-end: deployment sonrası servislerin doğru ayağa kalkması.

Checklist: Mobil uygulama ilk açılışta çökmedi · ENV yapılandırmaları doğru (API URL, analytics, config) · deployment sonrası health endpoint stabil

Mülakat: “Installation/deployment test, uygulamanın doğru kurulum başlatılabildiğini doğrular ve özellikle mobilde kritiktir.”

Configuration Test

Uygulamanın doğru environment ayarlarıyla çalıştığını doğrular. Eksik, boş veya yanlış formatlı ENV değerleri (API_URL, ANALYTICS_KEY, FEATURE_FLAGS) için otomatik validasyon koşulları; build sonrası health testleri yapılır.

Checklist: Tüm ENV değişkenleri tanımlı · API adresleri doğru ortamı gösteriyor · gizli anahtarlar eksiksiz · feature flag'ler doğru aktif/pasif durumda

Mülakat: “Configuration test, uygulamanın doğru environment ayarlarıyla sorunsuz çalıştığını doğrular; çoğu prod hatası aslında yanlış ENV kaynaklıdır.”

Compliance Test

Uygulamanın KVKK, GDPR, HIPAA gibi yasal ve düzenleyici gereksinimlere uygun çalıştığını doğrular. Loglama, anonimleştirme, veri saklama süreçleri incelenir.

Checklist: Kişisel veriler şifreli/anonim · kullanıcı verisi gereksiz saklanmıyor · yasal raporlama yükümlülükleri karşılanıyor

Mülakat: “Compliance test, uygulamanın KVKK/GDPR gibi yasal gereksinimlere uygunluğunu doğrular.”

Sistem ve Entegrasyon Testleri

Database & Migration Test

Veritabanı sorgularının, şema değişikliklerinin ve migration'ların doğru çalıştığını doğrular. Migration çalıştırılır, DB şeması doğrulanır; CRUD işlemleri gerçek test DB'de denetlenir; foreign key, index ve constraint kontrolleri yapılır.

Checklist: Migration sıfırdan kurulumda sorunsuz çalışıyor · eski veriyi bozmadan yükseltme yapılabilir · index ve FK yapıları doğru

Mülakat: “Database testleri ve migration testleri, veri bütünlüğünü ve şema değişikliklerinin güvenli uygulanmasını garanti eder.”

Interoperability Test

Uygulamanın ödeme servisleri, kimlik doğrulama, ERP/CRM gibi üçüncü parti servislerle uyumlu çalıştığını doğrular.

Checklist: Üçüncü parti API çağrıları doğru şema ile yapılıyor · rate limit ve hata yönetimi doğru

Mülakat: “Interoperability test, uygulamanın diğer servis ve sistemlerle sorunsuz etkileşime girebildiğini doğrular.”

Portability Test

Uygulamanın farklı platformlara veya altyapılara (Docker → Kubernetes, Linux → macOS) taşındığında doğru çalıştığını test eder.

Checklist: Ortam bağımlılıkları minimum · konfigürasyonlar environment tabanlı

Mülakat: “Portability test, uygulamanın farklı ortamlara sorunsuz taşınabildiğini doğrular.”

Maintainability Test

Kodun ne kadar kolay okunabilir, değiştirilebilir ve sürdürülebilir olduğunu ölçer: cyclomatic complexity, coupling, duplicate code ve dead code analizi.

Checklist: Fonksiyonlar küçük ve tek sorumluluklu · karmaşıklık metrikleri kabul edilebilir · ölü kod yok

En çok kullanılan üçlü: ESLint + Prettier (JS/TS) SonarQube/SonarCloud (genel) Madge + Lizard (dependency + complexity)

Mülakat: “Maintainability test, kodun uzun vadede ne kadar kolay geliştirilebileceğini ölçer.”

Statik Analiz Araçları

Ekosistem	Araçlar	Açıklama
JS / TS	ESLint, TSC strict, SonarQube	Complexity, duplication, dead code, maintainability rating
Java / Kotlin	SonarQube, PMD, Spot-Bugs, Android Lint	Bad practices, architectural violations, code smells
Python	Pylint, Flake8, MyPy, SonarQube	Styling, type errors, complexity, smells
Go	GolangCI-Lint, static-check	Hafif ve hızlı code quality kontrolü
C / C++	cppcheck, Clang-Tidy, SonarQube	Memory issues + complexity analizleri

Metrik, Duplicate ve Dependency Araçları

Araç	Ne Ölçer / Açıklama
SonarQube Maintainability	A–E arasında maintainability puanı; teknik borç ölçümü
Code Climate / Codacy	Maintainability, duplication, code smells, complexity metrikleri
Lizard	Cyclomatic complexity (dil bağımsız)
jscpd / PMD CPD	JS/TS ve Java için duplicate kod finder
Madge	Circular dependency graph çıkarır; modülerlik analizi
DepCheck (JS/TS)	Kullanılmayan bağımlılıklar ve hatalı importlar
NDepend (C#)	Coupling, dependency graph, architecture compliance
Webpack Analyzer	Bundle içi gereksiz paketlerin analizi

Refactoring, UI, Dokümantasyon ve Güvenlik Araçları

Araç	Açıklama
IntelliJ IDEA Inspections	Java/Kotlin için güçlü refactoring önerileri
Android Studio Lint	Android için lifecycle ve UI smell tespiti
React DevTools Profiler	State karmaşıklığı ve re-render performans analizi
Storybook + Visual Regression	UI component sürdürülebilirliği ve görsel tutarlılık
TypeDoc / JSDoc / Swagger	TypeScript, JavaScript ve backend API dokümantasyonu
GitHub CodeQL	Güvenlik + maintainability analizleri birlikte
Snyk / npm audit	Vulnerable dependency tespiti

Manuel vs Otomatik Test

Manuel: İnsan tarafından elle yapılır (butonlara tıklama, form doldurma, edge-case senaryoları deneme). Özellikle keşif testleri ve UX/görünüş testleri için önemlidir.

Otomatik: Kod veya script ile yazılır (unit, integration, API, E2E, performans testleri); CI/CD pipeline'ında otomatik koşar.

Mülakat: “Manuel testler keşif ve kullanıcı deneyimi için önemli; ancak sürdürülebilir güvence için unit, integration ve kritik E2E senaryolarını otomatik hale getirmek zorunludur.”